Aim: To integrate Firebase Cloud Firestore in a Flutter application and perform CRUD (Create, Read, Update, Delete) operations.

Theory:

Q1. What is Firebase Cloud Firestore?

Firebase Cloud Firestore is a fully-managed, cloud-hosted, NoSQL document-oriented database.

It provides real-time data synchronization, automatic multi-region replication, ACID transactions, and offline persistence, enabling responsive cross-platform applications with robust data consistency and horizontal scalability.

Q2. What is StreamBuilder in Flutter?

`StreamBuilder` is a stateful widget that asynchronously reconstructs its UI subtree in response to snapshots emitted by a Stream data source. It manages the widget lifecycle, automatically subscribing and unsubscribing to the stream, thereby facilitating reactive programming paradigms within Flutter's declarative framework.

Q3. Explain the Structure of Cloud Firestore.

Cloud Firestore organizes data hierarchically:

- Collections: Named containers that function as logical groupings for documents.
- Documents: Atomic units containing key-value pairs, supporting various data types.
- Subcollections: Collections embedded within documents, enabling deeply nested, scalable data structures without complex joins.

Q4. What are CRUD operations in Firestore?

CRUD operations represent the fundamental data manipulation interface:

- Create: Instantiate new documents via `set()` or `add()` methods.
- Read: Query or retrieve specific documents using `get()` or real-time `snapshots()`.
- Update: Modify existing document fields with `update()`.
- Delete: Permanently remove documents utilizing `delete()`.

Q5. What are the steps to integrate Firebase Cloud Firestore in Flutter?

1. The integration process involves:
2. Project Configuration: Register the application within the Firebase Console.
3. SDK Integration: Add the `cloud_firestore` dependency to `pubspec.yaml`.
4. Initialization: Execute `Firebase.initializeApp()` within the application's main context.
5. Implementation: Instantiate a `FirebaseFirestore` instance to execute database transactions.

Q7. Implement a flutter application to demonstrate CRUD operations in Firebase.

Code - (strawhat_model.dart):

```
class StrawHatModel {
  final String? id;
  final String name;
  final int age;
  final String role;
  final String alias;
  final int bounty;
  final String gender;
  final String imageAsset;

  StrawHatModel({
    this.id,
    required this.name,
    required this.age,
    required this.role,
    required this.alias,
    required this.bounty,
    required this.gender,
    required this.imageAsset,
  });

  Map<String, dynamic> toFirestore() {
    return {
      'name': name,
      'age': age,
      'role': role,
      'alias': alias,
      'bounty': bounty,
      'gender': gender,
      'imageAsset': imageAsset,
    };
  }

  factory
StrawHatModel.fromFirestore(Map<String,
dynamic> data, String documentId) {
    return StrawHatModel(
      id: documentId,
      name: data['name'] ?? '',
      age: data['age'] ?? 0,
      role: data['role'] ?? '',
      alias: data['alias'] ?? '',
      bounty: data['bounty'] ?? 0,
      gender: data['gender'] ?? '',
      imageAsset: data['imageAsset'] ??
'assets/images/luffy_wanted_poster.png',
    );
  }

  String get formattedBounty {
    if (bounty >= 1000000000) {
      final billions = bounty / 1000000000;
      return '${billions.toStringAsFixed(1)}B';
    } else if (bounty >= 1000000) {
      final millions = bounty / 1000000;
      return '${millions.toStringAsFixed(1)}M';
    } else {
      return bounty.toString();
    }
  }

  String get bountyWithSymbol {
    return '${formattedBounty} Berries';
  }
}
```

Code - (add_edit_crew_screen.dart):

```
import 'package:flutter/material.dart';
import '../services/firestore_service.dart';
import '../models/strawhat_model.dart';

class AddEditCrewScreen extends StatefulWidget
{
  final StrawHatModel? member;

  const AddEditCrewScreen({super.key,
this.member});

  @override
  State<AddEditCrewScreen> createState() =>
_AddEditCrewScreenState();
}

class _AddEditCrewScreenState extends
State<AddEditCrewScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController =
TextEditingController();
  final _ageController =
TextEditingController();
  final _roleController =
TextEditingController();
  final _aliasController =
TextEditingController();
  final _bountyController =
TextEditingController();
  final _imageAssetController =
TextEditingController();   String
_selectedGender = 'Male';

  final FirestoreService _firestoreService =
FirestoreService();
  bool _isLoading = false;

  final List<String> _genders = ['Male',
'Female'];

  final Map<String, String> _imageMap = {
    'luffy':
'assets/images/luffy_wanted_poster.png',
    'zoro':
'assets/images/zoro_wanted_poster.png',
    'nami':
'assets/images/nami_wanted_poster.png',
    'usopp':
'assets/images/usopp_wanted_poster.png',
```

```
    'sanji':
'assets/images/sanji_wanted_poster.png',
    'chopper':
'assets/images/chopper_wanted_poster.png',
    'robin':
'assets/images/robin_wanted_poster.png',
    'franky':
'assets/images/franky_wanted_poster.png',
    'brook':
'assets/images/brook_wanted_poster.png',
    'jinbe':
'assets/images/jinbe_wanted_poster.png',
  };

  @override
  void initState() {
    super.initState();
    if (widget.member != null) {
      _nameController.text =
widget.member!.name;
      _ageController.text =
widget.member!.age.toString();
      _roleController.text =
widget.member!.role;
      _aliasController.text =
widget.member!.alias;
      _bountyController.text =
widget.member!.bounty.toString();
      _selectedGender = widget.member!.gender;

      _imageAssetController.text =
widget.member!.imageAsset;
    }
  }

  @override
  void dispose() {
    _nameController.dispose();
    _ageController.dispose();
    _roleController.dispose();
    _aliasController.dispose();
    _bountyController.dispose();
    _imageAssetController.dispose();
    super.dispose();
  }

  String _getImageFromName(String name) {
    final key = name.toLowerCase().trim();
    return _imageMap[key] ??
'assets/images/luffy_wanted_poster.png';
  }

  Future<void> _saveMember() async {
    if (_formKey.currentState!.validate()) {
      setState(() => _isLoading = true);

      try {
        final age =
int.parse(_ageController.text);
        final bounty =
int.parse(_bountyController.text);
        final imageFromName =
_getImageFromName(_nameController.text);

        if (widget.member == null) {
          final newMember = StrawHatModel(
            name: _nameController.text.trim(),
            age: age,
            role: _roleController.text.trim(),
            alias:
_aliasController.text.trim(),
            bounty: bounty,
            gender: _selectedGender,

            imageAsset: imageFromName,
          );

          await
_firestoreService.addCrewMember(newMember);
        } else {
          final updatedData = {
            'name':
_nameController.text.trim(),
            'age': age,
            'role':
_roleController.text.trim(),
            'alias':
_aliasController.text.trim(),
            'bounty': bounty,
            'gender': _selectedGender,

            'imageAsset': imageFromName,
          };

          await _firestoreService

.updateCrewMember(widget.member!.id!,
updatedData);
        }

        if (mounted) Navigator.pop(context);
      } catch (e) {
        if (mounted) {

ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(
              content: Text('Error: $e'),
              backgroundColor: Colors.red,
            ),
          );
        }
      } finally {
        if (mounted) setState(() => _isLoading
= false);
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    final isEditing = widget.member != null;
    final imagePath =
_getImageFromName(_nameController.text);
```

```
    return Scaffold(
      appBar: AppBar(
        title: Text(isEditing ? 'Edit Crew
Member' : 'Add New Crew Member'),
        backgroundColor: Colors.orange,
        foregroundColor: Colors.white,
      ),
      body: _isLoading
          ? const Center(
        child: CircularProgressIndicator(color:
Colors.orange),
      )
          : Padding(
        padding: const EdgeInsets.all(24.0),
        child: Row(
          crossAxisAlignment:
CrossAxisAlignment.start,
          children: [

            Expanded(
              flex: 1,
              child: Container(
                height: 1000,
                decoration: BoxDecoration(
                  borderRadius:
BorderRadius.circular(16),
                  border: Border.all(color:
Colors.orange.shade200),
                ),
                child: ClipRRect(
                  borderRadius:
BorderRadius.circular(16),
                  child: Image.asset(
                    imagePath,
                    fit: BoxFit.cover,
                  ),
                ),
              ),
            ),

            const SizedBox(width: 40),

            Expanded(
              flex: 2,
              child: Form(
                key: _formKey,
                child: SingleChildScrollView(
                  child: Column(
                    children: [

                      // NAME
                      TextFormField(
                        controller:
_nameController,
                        decoration: const
InputDecoration(
                          labelText: 'Name',
                          border:
OutlineInputBorder(),
                        ),
                        onChanged: (_) =>
setState(() {}),
                        validator: (value) =>
                          value == null ||
value.isEmpty
                            ? 'Enter name'
                            : null,
                      ),
                      const SizedBox(height:
16),

                      // AGE
                      TextFormField(
                        controller:
_ageController,
                        keyboardType:
TextInputType.number,
                        decoration: const
InputDecoration(
                          labelText: 'Age',
                          border:
OutlineInputBorder(),
                        ),
                        validator: (value) =>
                          value == null ||
int.tryParse(value) == null
                            ? 'Enter valid age'
                            : null,
                      ),
                      const SizedBox(height:
16),

                      // ROLE
                      TextFormField(
                        controller:
_roleController,
                        decoration: const
InputDecoration(
                          labelText: 'Role',
                          border:
OutlineInputBorder(),
                        ),
                        validator: (value) =>
                          value == null ||
value.isEmpty
                            ? 'Enter role'
                            : null,
                      ),
                      const SizedBox(height:
16),

                      // ALIAS
                      TextFormField(
                        controller:
_aliasController,
                        decoration: const
InputDecoration(
                          labelText: 'Alias',
                          border:
OutlineInputBorder(),
                        ),
                        validator: (value) =>
                          value == null ||
value.isEmpty
                            ? 'Enter alias'
                            : null,
```

```
                      ),
                      const SizedBox(height:
16),

                      // BOUNTY
                      TextFormField(
                        controller:
_bountyController,
                        keyboardType:
TextInputType.number,
                        decoration: const
InputDecoration(
                          labelText: 'Bounty',
                          border:
OutlineInputBorder(),
                        ),
                        validator: (value) =>
                        value == null ||
int.tryParse(value) == null
                            ? 'Enter valid
bounty'
                            : null,
                      ),
                      const SizedBox(height:
16),

                      // GENDER

DropdownButtonFormField<String>(
                        value: _selectedGender,
                        decoration: const
InputDecoration(
                          labelText: 'Gender',
                          border:
OutlineInputBorder(),
                        ),
                        items: _genders
                            .map((g) =>
DropdownMenuItem(
                          value: g,
                          child: Text(g),
                        ))
                            .toList(),
                        onChanged: (val) =>
                            setState(() =>
_selectedGender = val!),
                      ),

                      const SizedBox(height:
30),

                      SizedBox(
                        width: double.infinity,
                        child: ElevatedButton(
                          onPressed:
_saveMember,
                          style:
ElevatedButton.styleFrom(
                            backgroundColor:
Colors.orange,
                            padding: const
EdgeInsets.symmetric(vertical: 18),
                          ),
                          child: Text(
                            isEditing
                                ? 'Update
Member'
                                : 'Add Member',
                            style: const
TextStyle(
                              fontWeight:
FontWeight.bold,
                              fontSize: 16,
                            ),
                          ),
                        ),
                      ),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ),
      );
    }
  }
```

Code - (crew_list_screen.dart):

```
import 'package:flutter/material.dart';
import '../services/firestore_service.dart';
import '../services/initial_data.dart';
import '../models/strawhat_model.dart';
import 'add_edit_crew_screen.dart';

class CrewListScreen extends StatefulWidget {
  const CrewListScreen({super.key});

  @override
  State<CrewListScreen> createState() =>
_CrewListScreenState();
}

class _CrewListScreenState extends
State<CrewListScreen> {
  final FirestoreService _firestoreService =
FirestoreService();
  final InitialDataService _initialDataService
= InitialDataService();
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _initializeData();
  }

  Future<void> _initializeData() async {
    await
_initialDataService.insertInitialDataIfNeeded()
;
  }

  Future<void> _resetCollection() async {
```

```
    setState(() => _isLoading = true);

    try {
      await
_firestoreService.deleteAllCrewMembers();

      if (mounted) {

ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text('All crew members
deleted successfully'),
            backgroundColor: Colors.orange,
          ),
        );
      }
    } catch (e) {
      if (mounted) {

ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text('Error deleting crew:
$e'),
            backgroundColor: Colors.red,
          ),
        );
      }
    } finally {
      if (mounted) {
        setState(() => _isLoading = false);
      }
    }
  }

  Future<void> _addInitialCrew() async {
    setState(() => _isLoading = true);

    try {
      await
_initialDataService.insertInitialDataIfNeeded()
;

      if (mounted) {

ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text('Initial crew members
added successfully'),
            backgroundColor: Colors.green,
          ),
        );
      }
    } catch (e) {
      if (mounted) {

ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text('Error adding crew:
$e'),
            backgroundColor: Colors.red,
          ),
        );
      }
    } finally {
      if (mounted) {
        setState(() => _isLoading = false);
      }
    }
  }

  void _showResetConfirmationDialog() {
    showDialog(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: const Text('Delete All Crew
Members'),
          content: const Text(
              'Are you sure you want to delete
ALL Straw Hat members? '
                  'This action cannot be
undone.'
          ),
          actions: [
            TextButton(
              onPressed: () =>
Navigator.pop(context),
              child: const Text('Cancel'),
            ),
            TextButton(
              onPressed: () {
                Navigator.pop(context);
                _resetCollection();
              },
              style:
TextButton.styleFrom(foregroundColor:
Colors.red),
              child: const Text('Delete All'),
            ),
          ],
        );
      },
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Straw Hat Pirates
Crew'),
        actions: [
          // Reset button
          IconButton(
            icon: const
Icon(Icons.delete_sweep),
            onPressed:
_showResetConfirmationDialog,
            tooltip: 'Delete all',
          ),
          // Add initial crew button
          IconButton(
            icon: const Icon(Icons.person_add),
            onPressed: _addInitialCrew,
            tooltip: 'Add initial crew',
          ),
```

```
        ],
      ),
      body: _isLoading
          ? const Center(
        child: CircularProgressIndicator(
          valueColor:
AlwaysStoppedAnimation<Color>(Colors.orange),
        ),
      )
          : _buildCrewGrid(),
      floatingActionButton:
FloatingActionButton(
        onPressed: () =>
_navigateToAddScreen(context),
        backgroundColor: Colors.orange,
        child: const Icon(Icons.add, color:
Colors.white),
      ),
    );
  }

  Widget _buildCrewGrid() {
    return StreamBuilder<List<StrawHatModel>>(
      stream:
_firestoreService.getCrewMembers(),
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return Center(
            child: Column(
              mainAxisAlignment:
MainAxisAlignment.center,
              children: [
                const Icon(Icons.error_outline,
size: 48, color: Colors.red),
                const SizedBox(height: 16),
                Text('Error:
${snapshot.error}'),
                const SizedBox(height: 16),
                ElevatedButton(
                  onPressed: _initializeData,
                  child: const Text('Retry'),
                ),
              ],
            ),
          );
        }

        if (snapshot.connectionState ==
ConnectionState.waiting) {
          return const Center(
            child: CircularProgressIndicator(
              valueColor:
AlwaysStoppedAnimation<Color>(Colors.orange),
            ),
          );
        }

        final crew = snapshot.data ?? [];

        if (crew.isEmpty) {
          return Center(
            child: Column(
              mainAxisAlignment:
MainAxisAlignment.center,
              children: [
                const Icon(
                  Icons.person_off,
                  size: 64,
                  color: Colors.grey,
                ),
                const SizedBox(height: 16),
                const Text(
                  'No crew members found',
                  style: TextStyle(fontSize:
18, color: Colors.grey),
                ),
                const SizedBox(height: 16),
                ElevatedButton(
                  onPressed: _addInitialCrew,
                  style:
ElevatedButton.styleFrom(
                    backgroundColor:
Colors.orange,
                  ),
                  child: const Text('Load
Initial Crew'),
                ),
              ],
            ),
          );
        }

        return GridView.builder(
          padding: const EdgeInsets.all(12),
          gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 5,
            childAspectRatio: 0.6,
            crossAxisSpacing: 8,
            mainAxisSpacing: 8,
          ),
          itemCount: crew.length > 10 ? 10 :
crew.length,
          itemBuilder: (context, index) {
            final member = crew[index];
            return _buildCrewCard(member);
          },
        );
      },
    );
  }

  Widget _buildCrewCard(StrawHatModel member) {
    return Card(
      elevation: 4,
      shape: RoundedRectangleBorder(
        borderRadius:
BorderRadius.circular(12),
      ),
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          Expanded(
            flex: 20,
```

```
            child: ClipRRect(
              borderRadius: const
BorderRadius.vertical(
                top: Radius.circular(12),
              ),
              child: Image.asset(
                member.imageAsset,
                fit: BoxFit.cover,
                width: double.infinity,
                errorBuilder: (context, error,
stackTrace) {
                  return Container(
                    color: Colors.grey[300],
                    child: Center(
                      child: Column(
                        mainAxisAlignment:
MainAxisAlignment.center,
                        children: [
                          Icon(
                            Icons.broken_image,
                            size: 40,
                            color:
Colors.grey[600],
                          ),
                          const
SizedBox(height: 8),
                          Text(
                            member.name[0],
                            style: const
TextStyle(
                              fontSize: 24,
                              fontWeight:
FontWeight.bold,
                            ),
                          ),
                        ],
                      ),
                    ),
                  );
                },
              ),
            ),
          ),

          // Name and Bounty section
          Expanded(
            flex: 4,
            child: Padding(
              padding: const
EdgeInsets.all(8.0),
              child: Column(
                crossAxisAlignment:
CrossAxisAlignment.start,
                mainAxisAlignment:
MainAxisAlignment.spaceBetween,
                children: [
                  // Name
                  Text(
                    member.name,
                    style: const TextStyle(
                      fontWeight:
FontWeight.bold,
                      fontSize: 14,
                    ),
                    maxLines: 2,
                    overflow:
TextOverflow.ellipsis,
                  ),

                  // Role (small text)
                  Text(
                    member.role,
                    style: TextStyle(
                      fontSize: 10,
                      color: Colors.grey[600],
                    ),
                    maxLines: 1,
                    overflow:
TextOverflow.ellipsis,
                  ),

                  // Bounty with styling
                  Container(
                    padding: const
EdgeInsets.symmetric(
                      horizontal: 6,
                      vertical: 3,
                    ),
                    decoration: BoxDecoration(
                      color:
Colors.orange.shade100,
                      borderRadius:
BorderRadius.circular(8),
                    ),
                    child: Row(
                      mainAxisSize:
MainAxisSize.min,
                      children: [
                        const Icon(

Icons.monetization_on,
                          size: 12,
                          color: Colors.orange,
                        ),
                        const SizedBox(width:
2),
                        Expanded(
                          child: Text(

member.formattedBounty,
                            style: TextStyle(
                              fontSize: 10,
                              fontWeight:
FontWeight.bold,
                              color:
Colors.orange.shade900,
                            ),
                            overflow:
TextOverflow.ellipsis,
                          ),
                        ),
                      ],
                    ),
                  ),

                  // Edit and Delete buttons
```

```
                  Row(
                    mainAxisAlignment:
MainAxisAlignment.end,
                    children: [
                      IconButton(
                        icon: const
Icon(Icons.edit, size: 16),
                        color: Colors.blue,
                        onPressed: () =>
_navigateToEditScreen(context, member),
                        constraints: const
BoxConstraints(),
                        padding: const
EdgeInsets.all(2),
                      ),
                      IconButton(
                        icon: const
Icon(Icons.delete, size: 16),
                        color: Colors.red,
                        onPressed: () =>
_showDeleteDialog(context, member),
                        constraints: const
BoxConstraints(),
                        padding: const
EdgeInsets.all(2),
                      ),
                    ],
                  ),
                ],
              ),
            ),
          ),
        ),
      ],
    ),
  );
}

void _navigateToAddScreen(BuildContext
context) {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => const
AddEditCrewScreen(),
    ),
  );
}

void _navigateToEditScreen(BuildContext
context, StrawHatModel member) {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) =>
AddEditCrewScreen(member: member),
    ),
  );
}

void _showDeleteDialog(BuildContext context,
StrawHatModel member) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Delete Crew
Member'),
      content: Text('Are you sure you want to
remove ${member.name} from the crew?'),
      actions: [
        TextButton(
          onPressed: () =>
Navigator.pop(context),
          child: const Text('Cancel'),
        ),
        TextButton(
          onPressed: () async {
            Navigator.pop(context);
            await
_firestoreService.deleteCrewMember(member.id!);
            if (mounted) {

ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content:
Text('${member.name} removed from crew'),
                  backgroundColor:
Colors.red,
                ),
              );
            }
          },
          style:
TextButton.styleFrom(foregroundColor:
Colors.red),
          child: const Text('Delete'),
        ),
      ],
    ),
  );
}
}
```

Code - (firestore_service.dart):

```
import
'package:cloud_firestore/cloud_firestore.dart';
import '../models/strawhat_model.dart';

class FirestoreService {
  final CollectionReference _crewCollection =

FirebaseFirestore.instance.collection('straw_ha
t_pirates');

  Future<void> addCrewMember(StrawHatModel
member) async {
    try {
      await
_crewCollection.add(member.toFirestore());
    } catch (e) {
      print('Error adding crew member: $e');
      rethrow;
    }
  }
```

```
  Stream<List<StrawHatModel>> getCrewMembers()
{
    return _crewCollection
        .orderBy('bounty', descending: true)
        .snapshots()
        .map((snapshot) {
      return snapshot.docs.map((doc) {
        return StrawHatModel.fromFirestore(
            doc.data() as Map<String, dynamic>,
            doc.id
        );
      }).toList();
    });
  }

  Future<StrawHatModel?> getCrewMember(String
id) async {
    try {
      DocumentSnapshot doc = await
_crewCollection.doc(id).get();
      if (doc.exists) {
        return StrawHatModel.fromFirestore(
            doc.data() as Map<String, dynamic>,
            doc.id
        );
      }
      return null;
    } catch (e) {
      print('Error getting crew member: $e');
      return null;
    }
  }

  Future<void> updateCrewMember(String id,
Map<String, dynamic> updatedData) async {
    try {
      await
_crewCollection.doc(id).update(updatedData);
    } catch (e) {
      print('Error updating crew member: $e');
      rethrow;
    }
  }

  Future<void> deleteCrewMember(String id)
async {
    try {
      await _crewCollection.doc(id).delete();
    } catch (e) {
      print('Error deleting crew member: $e');
      rethrow;
    }
  }

  Future<void> deleteAllCrewMembers() async {
    try {
      final QuerySnapshot snapshot = await
_crewCollection.get();
      final WriteBatch batch =
FirebaseFirestore.instance.batch();

      for (var doc in snapshot.docs) {
        batch.delete(doc.reference);
      }

      await batch.commit();
      print('All crew members deleted
successfully');
    } catch (e) {
      print('Error deleting all crew members:
$e');
      rethrow;
    }
  }

  Stream<List<StrawHatModel>>
getCrewMembersByField(String field, dynamic
value) {
    return _crewCollection
        .where(field, isEqualTo: value)
        .orderBy('bounty', descending: true)
        .snapshots()
        .map((snapshot) {
      return snapshot.docs.map((doc) {
        return StrawHatModel.fromFirestore(
            doc.data() as Map<String, dynamic>,
            doc.id
        );
      }).toList();
    });
  }
}
```

Code - (initial_data.dart):

```
import
'package:cloud_firestore/cloud_firestore.dart';
import '../models/strawhat_model.dart';

class InitialDataService {
  final CollectionReference _crewCollection =

FirebaseFirestore.instance.collection('straw_ha
t_pirates');

  final List<StrawHatModel> initialCrew = [
    StrawHatModel(
      name: 'Monkey D. Luffy',
      age: 19,
      role: 'Captain',
      alias: 'Straw Hat Luffy',
      bounty: 3000000000, // 3 Billion
      gender: 'Male',
      imageAsset:
'assets/images/luffy_wanted_poster.png',
    ),
    StrawHatModel(
      name: 'Roronoa Zoro',
      age: 21,
      role: 'Swordsman / First Mate',
      alias: 'Pirate Hunter Zoro',
      bounty: 1111000000, // 1.111 Billion
      gender: 'Male',
      imageAsset:
'assets/images/zoro_wanted_poster.png',
    ),
```

```
    StrawHatModel(
      name: 'Nami',
      age: 20,
      role: 'Navigator',
      alias: 'Cat Burglar Nami',
      bounty: 366000000, // 366 Million
      gender: 'Female',
      imageAsset:
'assets/images/nami_wanted_poster.png',
    ),
    StrawHatModel(
      name: 'Usopp',
      age: 19,
      role: 'Sniper',
      alias: 'God Usopp',
      bounty: 500000000, // 500 Million
      gender: 'Male',
      imageAsset:
'assets/images/usopp_wanted_poster.png',
    ),
    StrawHatModel(
      name: 'Sanji',
      age: 21,
      role: 'Cook',
      alias: 'Black Leg Sanji',
      bounty: 1032000000, // 1.032 Billion
      gender: 'Male',
      imageAsset:
'assets/images/sanji_wanted_poster.png',
    ),
    StrawHatModel(
      name: 'Tony Tony Chopper',
      age: 17,
      role: 'Doctor',
      alias: 'Cotton Candy Lover Chopper',
      bounty: 1000, // 1000 Berries
      gender: 'Male',
      imageAsset:
'assets/images/chopper_wanted_poster.png',
    ),
    StrawHatModel(
      name: 'Nico Robin',
      age: 30,
      role: 'Archaeologist',
      alias: 'Devil Child',
      bounty: 930000000, // 930 Million
      gender: 'Female',
      imageAsset:
'assets/images/robin_wanted_poster.png',
    ),
```

…add franky, brook, jinbe…

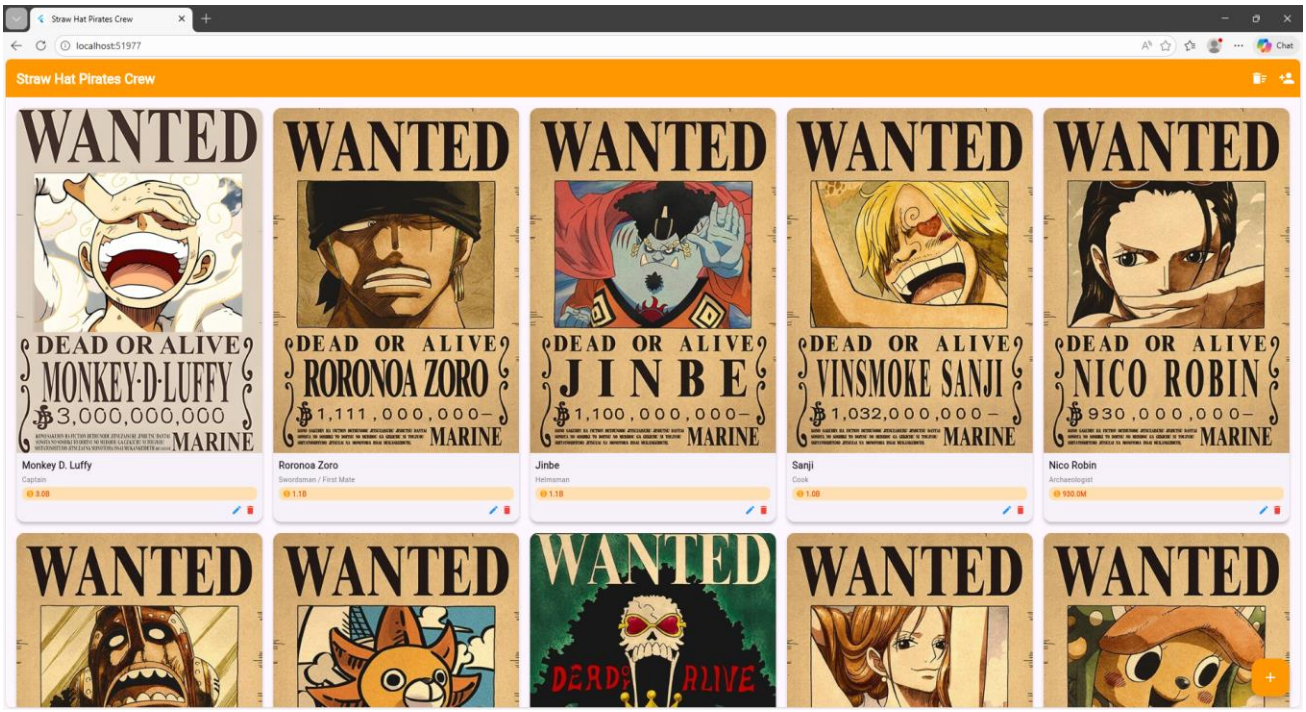
```
  ];

  Future<void> insertInitialDataIfNeeded()
async {
    try {
      final snapshot = await
_crewCollection.limit(1).get();
      if (snapshot.docs.isEmpty) {
        for (var member in initialCrew) {
          await
_crewCollection.add(member.toFirestore());
        }
        print('Initial 10 Straw Hat members
added successfully!');
      } else {
        print('Crew members already exist in
database');
      }
    } catch (e) {
      print('Error inserting initial data:
$e');
    }
  }
}
```

Code - (main.dart):

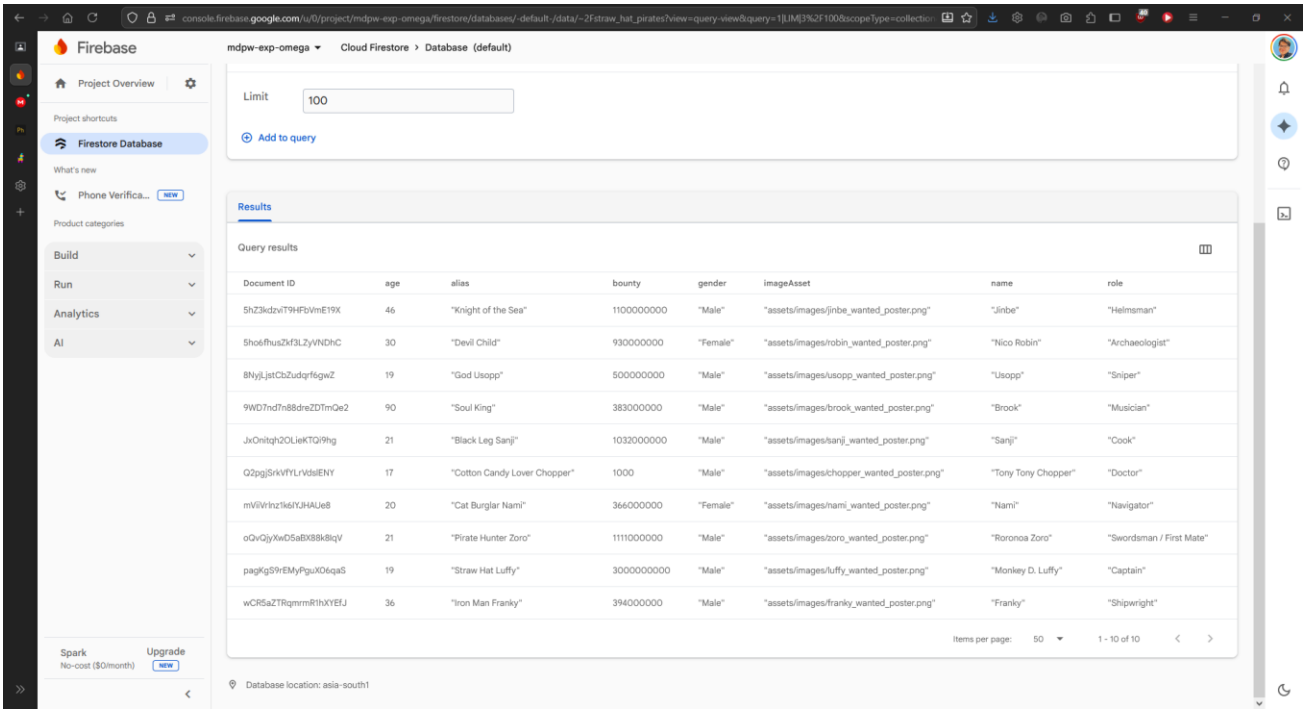
```
import 'package:flutter/material.dart';
import
'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
import 'screens/crew_list_screen.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options:
DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Straw Hat Pirates Crew',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.orange,
        useMaterial3: true,
        appBarTheme: const AppBarTheme(
          backgroundColor: Colors.orange,
          foregroundColor: Colors.white,
          titleTextStyle: TextStyle(
            color: Colors.white,
            fontSize: 20,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
      home: const CrewListScreen(),
    );
  }
}
```

Output:

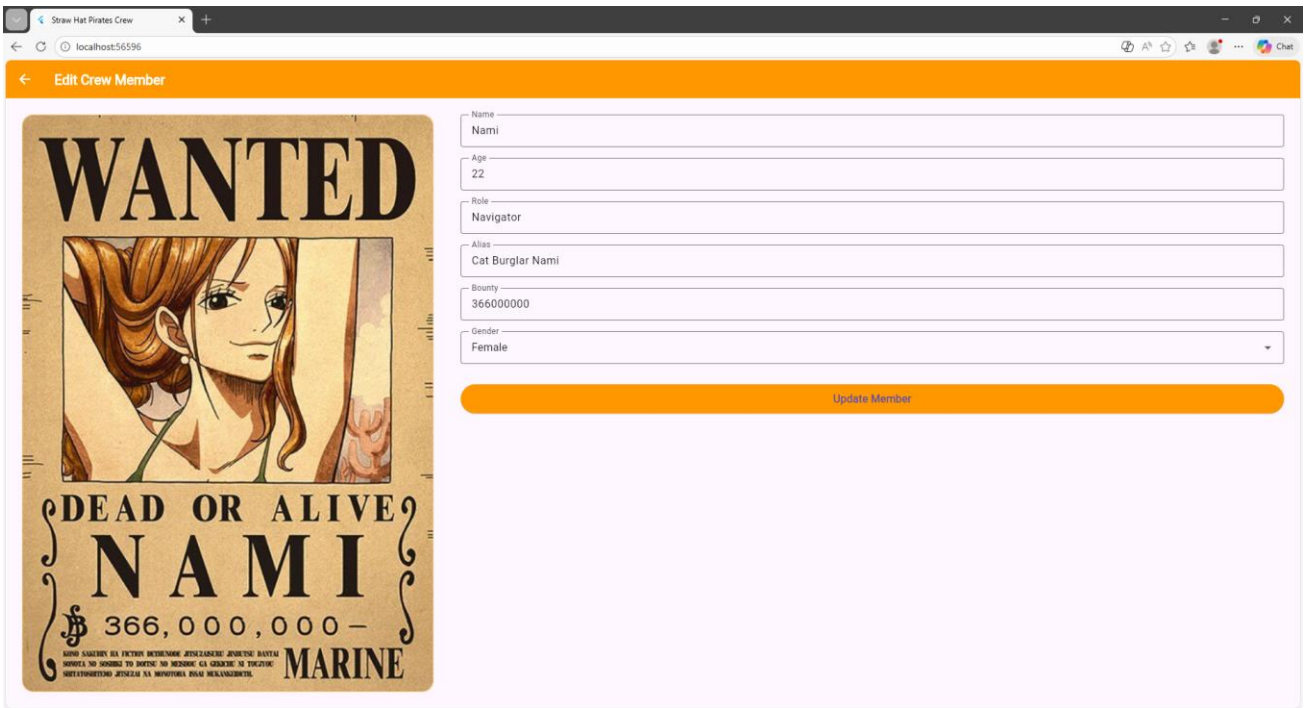
Initial Data loaded in Web interface displaying Straw Hat Pirates crew members in a grid layout with wanted posters, bounty details, and edit/delete options.



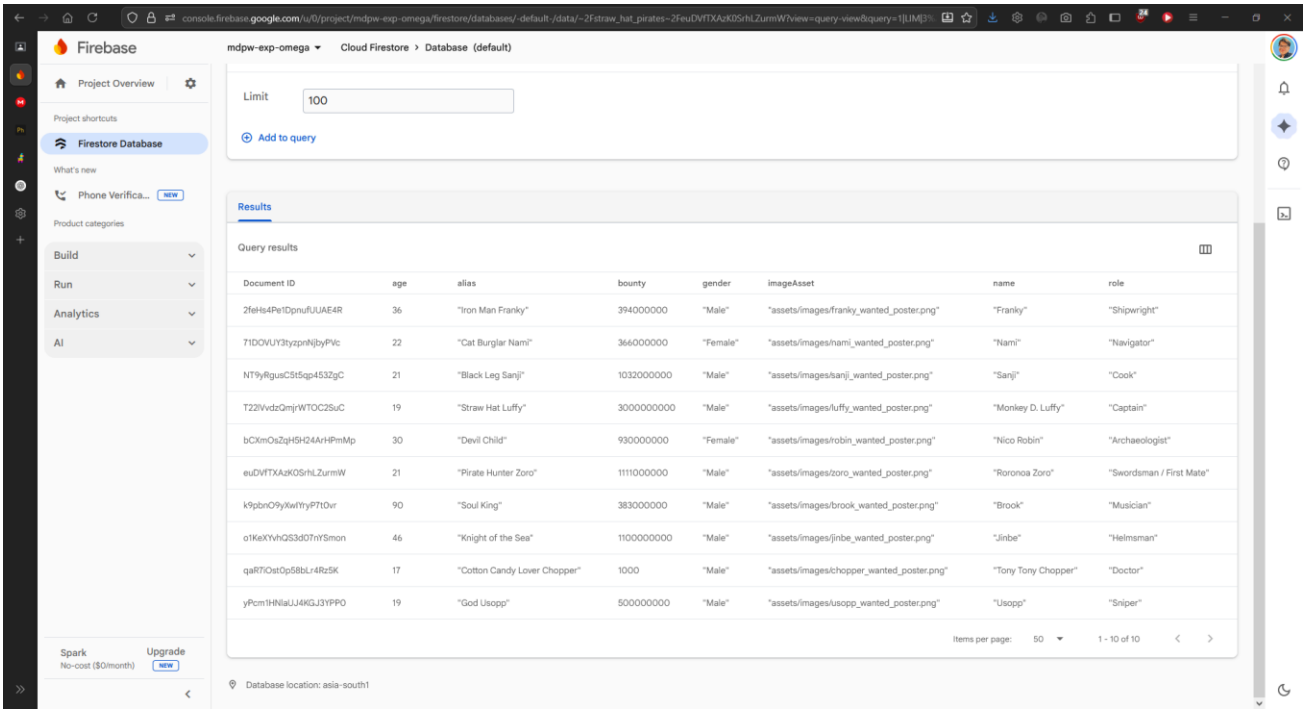
Initial Data inserted into firebase in 'straw_hat_pirates' table. (Nami's age is 20)



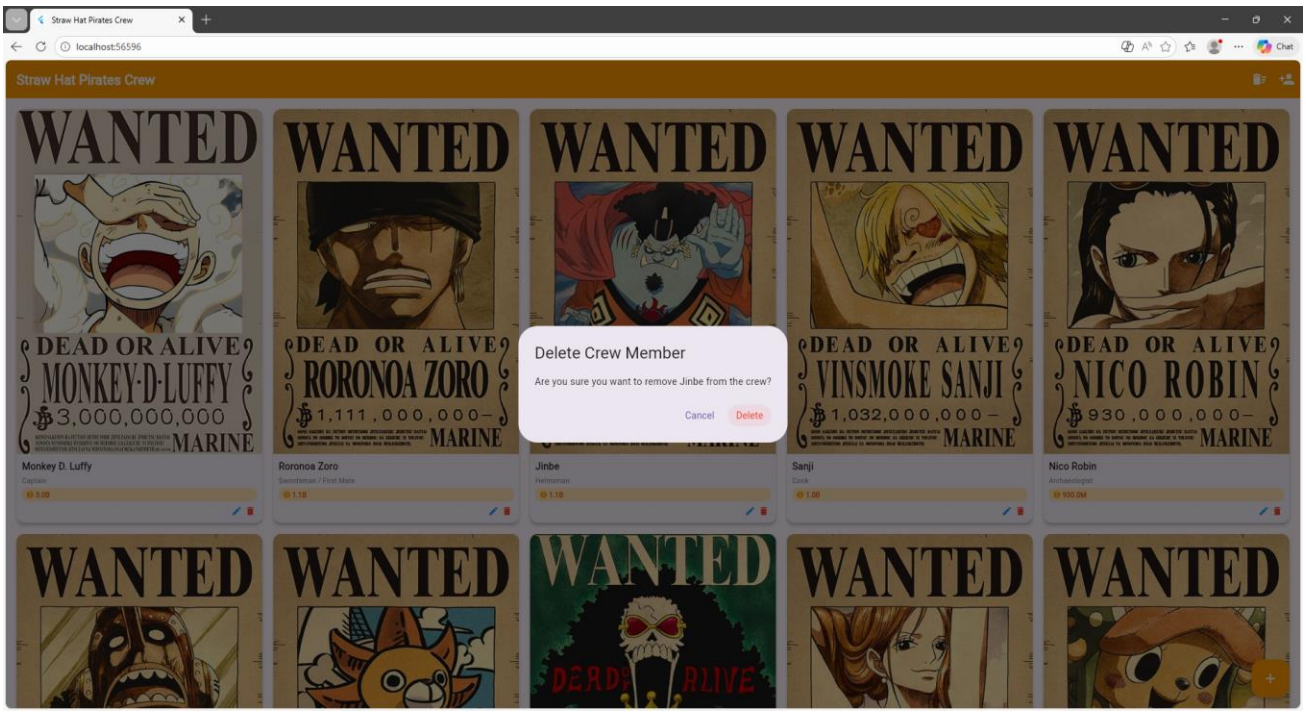
Editing Nami's age to 22 to perform demonstrate update operation



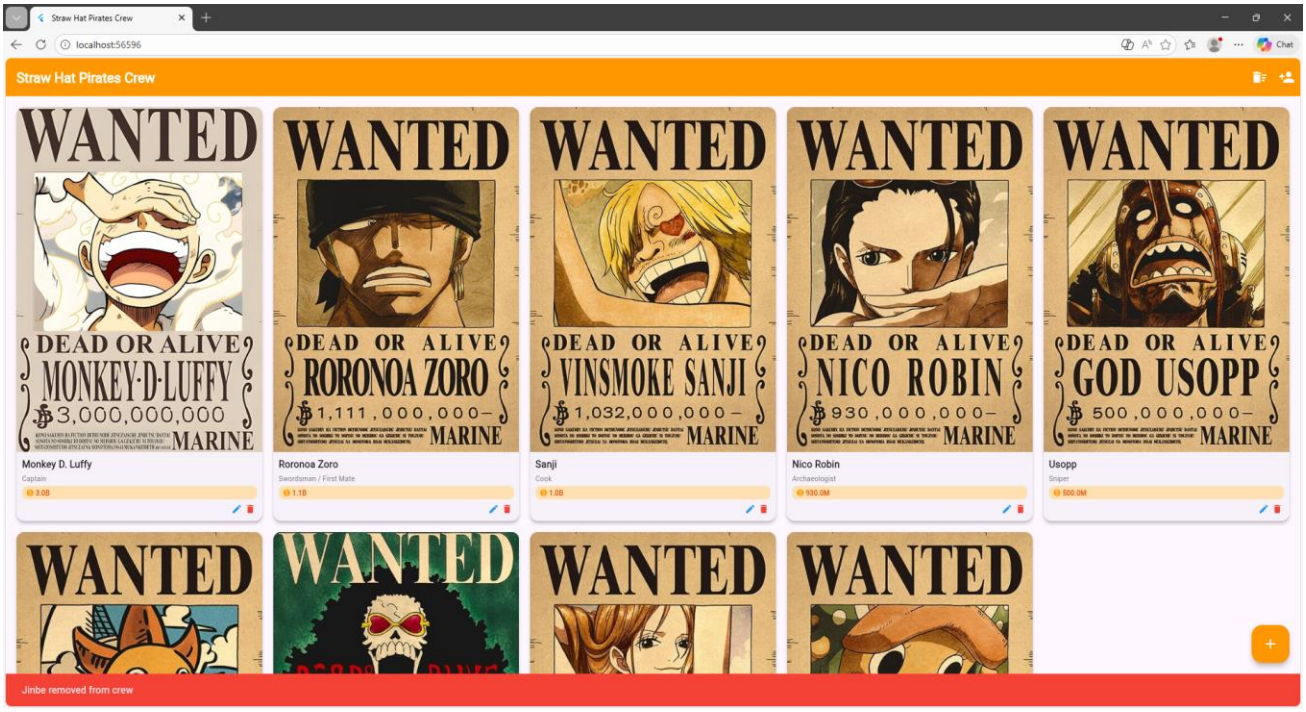
Successfully Nami's age updated to 22 in the firebase database table



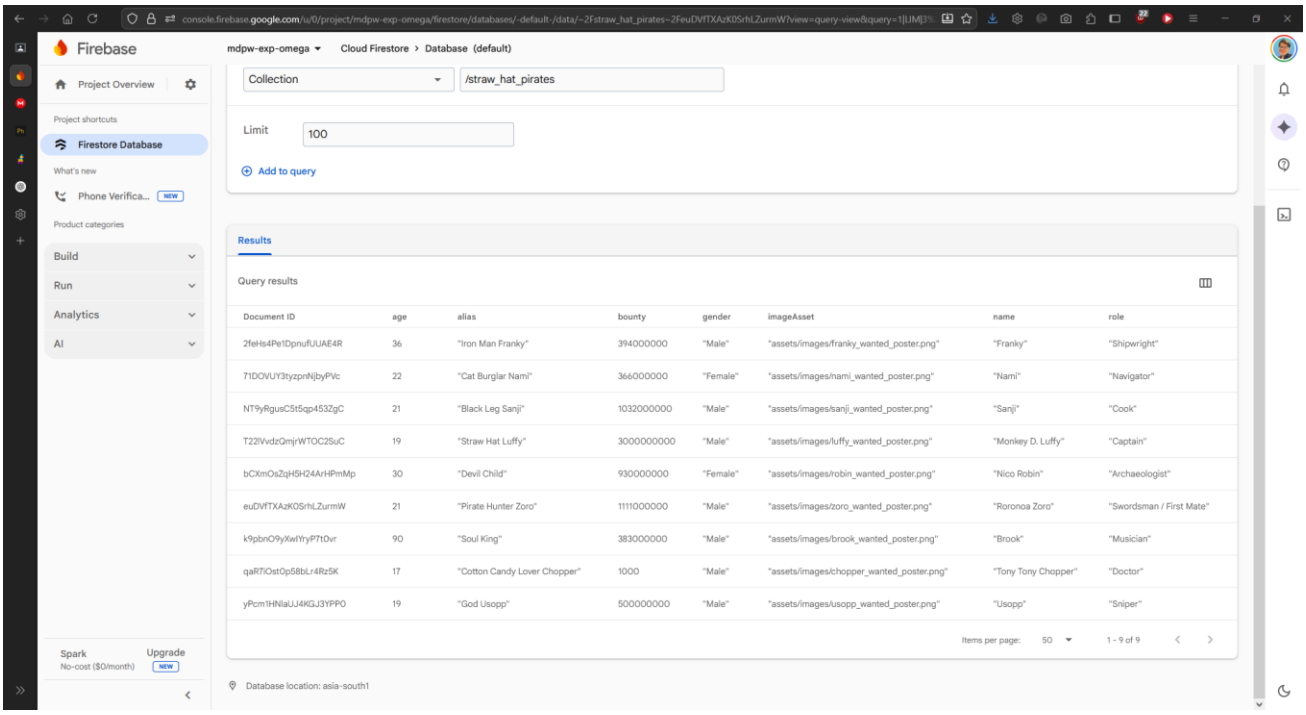
Deleting Jinbe to demonstrate delete operation



Successfully deleted Jinbe from web interface



Successfully Deleted Jinbe from the firebase database table



Conclusion

Successfully integrating Cloud Firestore with Flutter and demonstrated CRUD operations to provide a robust, scalable backend solution for efficient and real-time data management.